

COM2002-1 Programación de Software de Sistemas**Profesor :** Joaquín Soto**Ayudante:** Arnaldo González

Ayudantía 4

Punteros, strings, memoria dinámica y estructuras de datos

P1. Preguntas conceptuales. Responda de forma breve y justifique cuando corresponda.

- Explique la diferencia entre declarar `'char *p'` y `'char arreglo[20]'`. Indique qué representa cada uno y qué ocurre al usar `'sizeof'` sobre ellos dentro de la función donde fueron declarados.
- Considere la declaración `'const char *s'`. ¿Qué se puede modificar a través de `'s'` y qué no? Dé un ejemplo de operación válida y una inválida.
- Un estudiante propone invertir una cadena reservando un segundo arreglo del mismo tamaño. Explique por qué la inversión puede hacerse en el mismo arreglo y cuál sería la complejidad temporal y espacial de una solución que use dos índices y un intercambio de caracteres.
- En un árbol binario que representa una expresión, las hojas almacenan valores y los nodos internos almacenan operaciones. Explique qué condición de término usaría en una función recursiva de evaluación y qué debería retornar en ese caso.

P2. Se requiere construir una función que una dos strings **sin modificar ninguna de las cadenas originales**. La función debe reservar memoria dinámica para almacenar el resultado completo y retornar un puntero al nuevo string.

Implemente:

```
char *unir_cadenas(const char *a, const char *b);
```

Para simplificar, puede asumir que ambas entradas son strings válidos terminados en `'\0'` y que las reservas de memoria necesarias siempre tienen éxito.

Su función debe cumplir con lo siguiente:

- calcular cuánto espacio se necesita para el resultado;
- solicitar la memoria correspondiente con `'malloc'`;
- copiar primero `'a'` y luego `'b'`;
- dejar exactamente un carácter `'\0'` al final;
- retornar la dirección de la nueva cadena.

P3. Escriba una función que rote una cadena una posición hacia la izquierda **en el mismo arreglo**, sin crear otro arreglo del tamaño de la cadena.

Por ejemplo:

```
"computacion"  -> "omputacionc"
```

Implemente:

```
void rotar_izquierda(char *s);
```

Puede usar una o más variables auxiliares simples (por ejemplo, un 'char' temporal), pero no puede reservar un segundo arreglo para copiar la cadena completa. Explique brevemente la idea utilizada.

P4. Estructuras en C: concepto y uso. Responda de forma breve y use ejemplos pequeños cuando corresponda.

- a) Explique qué es una 'struct' en C y cuál es su utilidad. Mencione una situación en la que resulte más conveniente agrupar varios datos dentro de una estructura que utilizar variables independientes.
- b) Considere la siguiente definición:

```
typedef struct {  
    char nombre[30];  
    int edad;  
} Persona;
```

Muestre cómo declarar una variable llamada 'p' de tipo 'Persona' y cómo asignar un valor a su campo 'edad'.

- c) Explique la diferencia entre acceder a un campo mediante '.' y mediante '->'. ¿En qué caso se utiliza cada operador? Incluya un ejemplo pequeño.
- d) Explique por qué podría ser útil pasar un puntero a una 'struct' a una función en lugar de pasar la estructura completa. Mencione al menos una ventaja y un posible ejemplo de uso.
- e) Imagine una estructura que represente un producto de una tienda, con campos como código, precio y stock. Sin implementar nada en profundidad, indique qué tipo de información podría almacenarse allí y mencione una función que podría recibir un puntero a esa estructura para modificar alguno de sus datos.
- f) Suponga que se quiere almacenar información de 100 estudiantes usando la estructura 'Persona'. ¿Qué debería declararse para guardar todas las personas y cómo accedería, por ejemplo, a la edad del estudiante ubicado en la posición 7?

- g) Muestre una forma sencilla de inicializar una variable de tipo 'Persona' al momento de declararla. Explique brevemente qué ventaja tiene inicializar una estructura desde el comienzo en lugar de dejar sus campos sin definir.
- h) ¿Qué diferencia conceptual existe entre declarar una estructura directamente y usar 'typedef' para darle un nombre de tipo? No es necesario escribir una definición completa; basta con explicar para qué sirve 'typedef' en este contexto.
- i) Suponga que se reserva memoria dinámicamente para una variable de tipo 'Persona' y se obtiene un puntero 'Persona *p'. ¿Cómo accedería al campo 'edad' y qué operador utilizaría? Explique brevemente por qué no se utiliza '.' en este caso.
- j) Considere una función 'void cambiar_edad(Persona *p)'. Si dentro de la función se ejecuta 'p->edad = 30', ¿qué ocurre con la variable original que fue entregada a la función? Explique por qué el cambio puede observarse fuera de la función.
- k) ¿Qué diferencia habría entre trabajar con 'Persona p' y trabajar con 'Persona *p'? Mencione qué almacena cada variable y cómo se accederían sus campos.
- l) Suponga que se tiene un arreglo de 'Persona' y un puntero que apunta a una de sus posiciones. ¿Qué ventaja puede tener recorrer el arreglo usando un puntero en lugar de utilizar directamente el índice? No es necesario escribir código.
- m) Si se reserva memoria con 'malloc' para una estructura 'Persona', ¿qué debería hacerse con esa memoria cuando ya no sea necesaria? Explique qué función de C se utiliza y por qué es importante hacerlo.
- n) Una 'struct' puede contener otra 'struct' como uno de sus campos. Mencione un ejemplo sencillo de este tipo de organización y explique brevemente por qué podría ser útil.
- ñ) ¿Qué problema podría aparecer si se trabaja con un puntero a una estructura que no apunta a una posición válida de memoria? Mencione una consecuencia posible y cómo podría evitarse conceptualmente.